

Knowledge Bytes - Level 2 E-Commerce

FILE: main.py

Byte 1: FastAPI app creation and router wiring

Builds on: None — starting point

In plain terms:

This is the backend's front door. It creates the web app and plugs in all feature sections.

The code:

```
app = FastAPI(title="Level 2 E-Commerce Order Management & Search API",
              version="2.0.0", lifespan=lifespan)
```

What's happening:

Creates a single shared FastAPI instance with a title, version, and startup lifespan hook. Everything else mounts onto this object.

Why it matters:

Without this object there is no server. It is the anchor for routing, middleware, and startup logic.

Byte 2: Lifespan ensures Postgres tables exist

Builds on: Byte 1

In plain terms:

On every startup, make sure the order tables exist before accepting traffic.

The code:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    import app.models.models # noqa: F401 (register PG models)
    Base.metadata.create_all(bind=engine)
    logger.info("Ensured PostgreSQL tables exist (users, orders, order_items)")
    yield
```

What's happening:

Imports ORM models so SQLAlchemy knows them, then creates missing tables. `yield` means run the app after this setup.

Why it matters:

Prevents crashes on first boot against an empty Postgres. It is idempotent and safe on every restart.

Byte 3: CORS, routers, and health check

Builds on: Byte 1

In plain terms:

Let the browser frontend call the API, expose all endpoints, and provide an aliveness probe.

The code:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:5173", "http://127.0.0.1:5173",
        "http://localhost:8080", "http://127.0.0.1:8080",
        "http://localhost", "http://127.0.0.1",
    ],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

What's happening:

Does CORS permits listed frontend origins. Separately, `include\_router` calls mount users, products, orders, and search, and `/api/health` returns `{ "status": "ok" }`.

Why it matters:

Without CORS the Vue app is blocked by the browser. Without router mounting, endpoint files exist but are unreachable.

PUTTING IT TOGETHER

`main.py` boots one FastAPI app, ensures Postgres tables exist at startup, opens CORS for Vite and nginx origins, mounts four routers, and exposes health. It stays thin and delegates all real logic outward.

FILE: config.py

Byte 1: Single Settings object for all infrastructure

Builds on: None — starting point

In plain terms:

One place that reads every connection string from the environment with local fallbacks.

The code:

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")
    DATABASE_URL: str = "postgresql+psycopg2://ecom:ecom123@localhost:5435/ecom_orders"
    MONGO_URL: str = "mongodb://localhost:27017"
    MONGO_DB: str = "ecom"
```

What's happening:

`BaseSettings` auto-loads matching env vars, falling back to defaults. It covers Postgres, MongoDB, RabbitMQ/Celery, and Elasticsearch in one class.

Why it matters:

Every module imports `settings` instead of hardcoding URLs. Changing environments means changing `.env`, not code.

Byte 2: Sync tuning and canonical statuses

Builds on: Byte 1

In plain terms:

Controls retry and polling behavior, and fixes the only legal order states.

The code:

```
ORDER_STATUSES = ("PENDING", "PROCESSING", "SHIPPED")
def normalize_status(value: str) -> str:
    return (value or "").strip().upper()
```

What's happening:

Defines queue name, retry count, base delay, and poll window elsewhere in the file, plus the exact uppercase status tuple. `normalize\_status` uppercases user input.

Why it matters:

Prevents drift like `pending` vs `PENDING` breaking filters and search facets. Poll and retry values decide freshness versus load.

PUTTING IT TOGETHER

`config.py` centralizes URLs, sync knobs, and the canonical status vocabulary. Importing `settings`, `ORDER\_STATUSES`, and `normalize\_status` keeps Postgres, Elasticsearch, and validation consistent.

FILE: database.py

Byte 1: Engine, session factory, and Base

Builds on: None — starting point

In plain terms:

Opens the connection to the transactional orders database. `PostgreSQL stores ONLY: users, orders, order\_items.`

The code:

```
engine = create_engine(settings.DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
Base = declarative_base()
```

What's happening:

Engine pools connections with health checks. `SessionLocal` makes per-request sessions with explicit commits. `Base` is the parent for all ORM models.

Why it matters:

Postgres is the source of truth for orders. Every order read and write flows through this engine.

Byte 2: Request-scoped session dependency

Builds on: Byte 1

In plain terms:

Give each API request its own database session, then always close it.

The code:

```
def get_db():
    """FastAPI dependency yielding a request-scoped DB session."""
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

What's happening:

FastAPI injects this via `Depends(get\_db)`. It yields a fresh session and closes it afterward, even on errors.

Why it matters:

Prevents connection leaks and cross-request contamination. Cleanup is guaranteed by `finally`.

PUTTING IT TOGETHER

`database.py` owns the Postgres connection, session factory, declarative base, and `get\_db` dependency. It knows nothing about products, which intentionally live in MongoDB.

FILE: mongo.py

Byte 1: Lazy singleton Mongo client

Builds on: None — starting point

In plain terms:

Connect to the product catalog database once and reuse that connection.

The code:

```
_client: Optional[MongoClient] = None
def get_mongo_client() -> MongoClient:
    global _client
    if _client is None:
        _client = MongoClient(settings.MONGO_URL, serverSelectionTimeoutMS=5000)
    return _client
```

What's happening:

Caches the `MongoClient` globally on first use. Later calls return the same client instead of reconnecting.

Why it matters:

Avoids connection churn on every catalog request. `MongoClient` internally pools, so one shared instance is correct.

Byte 2: Products collection accessor and test reset

Builds on: Byte 1

In plain terms:

A helper to get the products table, plus a way to drop the cached client in tests.

The code:

```
def get_products_collection() -> Collection:
    client = get_mongo_client()
    return client[settings.MONGO_DB][settings.MONGO_PRODUCTS_COLLECTION]
```

What's happening:

Selects the configured database and `products` collection. `reset\_mongo\_client` closes and clears the cache so tests can re-point settings.

Why it matters:

Callers never hardcode db or collection names. Tests can swap to `mongomock` without restarting the process.

PUTTING IT TOGETHER

`mongo.py` lazily owns one Mongo client, exposes the products collection, and provides a reset hook. It is the catalog counterpart to `database.py` and serves storefront, admin, and checkout validation.

FILE: search.py

Byte 1: Orders index mapping as projection

Builds on: None — starting point

In plain terms:

Defines how Elasticsearch stores a flattened copy of an order for admin search. `Elasticsearch client + index management (search projection only).`

The code:

```
ORDER_INDEX_MAPPING = {
    "mappings": {
        "properties": {
            "order_id": {"type": "integer"},
            "status": {"type": "keyword"},
            "total_amount": {"type": "float"},
```

What's happening:

Declares field types: exact `keyword` for status filtering, `text` with keyword subfields for customer and titles, `nested` for items, dates for ranges.

Why it matters:

Mapping decides what queries work. Keyword enables exact status facets; text enables full-text search; nested keeps line items queryable.

Byte 2: Lazy client and index lifecycle

Builds on: Byte 1

In plain terms:

Connect once, create the index if missing, delete it when rebuilding.

The code:

```
def ensure_order_index() -> None:
    """Create the orders index if missing (idempotent)."""
    es = get_es_client()
    if not es.indices.exists(index=settings.ELASTICSEARCH_INDEX):
        es.indices.create(index=settings.ELASTICSEARCH_INDEX, body=ORDER_INDEX_MAPPING)
```

What's happening:

`get\_es\_client` caches one `Elasticsearch` instance. `ensure\_order\_index` creates the index only if absent; `delete\_order\_index` drops it for seed and reindex.

Why it matters:

Makes boot and sync safe to retry. Rebuilds depend on delete-then-create without manual index management.

PUTTING IT TOGETHER

`core/search.py` defines the order search schema and owns the client plus create and delete helpers. It never touches Postgres or Mongo directly; it only hosts the projection others write and query.

FILE: messaging.py

Byte 1: Publish sync hint after commit

Builds on: None — starting point

In plain terms:

After Postgres is safely committed, tell the background worker which order to copy to search. `PostgreSQL is the source of truth.`

The code:

```
def publish_order_sync(order_id: int) -> bool:
    try:
        from app.services.sync import sync_order
        sync_order.apply_async(args=[order_id])
        logger.info("Published sync task for order %s", order_id)
        return True
```

What's happening:

Enqueues a Celery task carrying only the order id. The worker re-reads fresh rows from Postgres; no order data travels via the queue.

Why it matters:

Keeps the request fast and decoupled. Passing only the id avoids stale data if the order changes before the worker runs.

Byte 2: Broker failure never rolls back Postgres

Builds on: Byte 1

In plain terms:

If RabbitMQ is down, log it and let the periodic poller backfill later.

The code:

```
except Exception: # noqa: BLE001 - broker down must not break the request
    logger.exception("Failed to publish sync task for order %s", order_id)
    return False
```

What's happening:

Swallows publish errors and returns `False`. Callers log a warning but keep the committed order. Strategy B polling picks it up.

Why it matters:

Checkout must succeed even when search infra is down. This is what makes dual-write safe instead of fragile.

PUTTING IT TOGETHER

`messaging.py` is the narrow bridge from request to worker: enqueue the id on success, return True or False, never undo Postgres. It enables Strategy A while relying on polling as backup.

FILE: celery_app.py

Byte 1: Celery app on RabbitMQ with safe delivery

Builds on: None — starting point

In plain terms:

Creates the background task system that carries order-sync jobs. `Celery application: RabbitMQ broker + 15s polling beat schedule.`

The code:

```
celery = Celery("order_sync", broker=settings.CELERY_BROKER_URL, backend="rpc://")
celery.conf.update(
    task_acks_late=True,
```

```
task_reject_on_worker_lost=True,  
worker_prefetch_multiplier=1,
```

What's happening:

Points Celery at RabbitMQ. Late acks plus reject-on-lost means a crashed worker requeues the job instead of losing it. Prefetch 1 spreads work fairly.

Why it matters:

Guarantees an order sync is not silently dropped during worker crashes. This is the transport for dual-write.

Byte 2: Queue routing and periodic polling schedule

Builds on: Byte 1

In plain terms:

Send sync tasks to one queue and run a safety-net sweep every 15 seconds.

The code:

```
beat_schedule={  
    # Strategy B: periodic polling bulk safety net, ~every 15 seconds.  
    "poll-changed-orders": {  
        "task": "app.services.sync.poll_changed_orders",  
        "schedule": 15.0,  
    },  
},
```

What's happening:

Routes sync tasks to the `order\_sync` queue with a rate limit, and registers a beat entry that fires `poll\_changed\_orders` every 15 seconds.

Why it matters:

Implements Strategy B alongside Strategy A. Even lost broker messages get bulk-synced within seconds.

PUTTING IT TOGETHER

`celery\_app.py` configures reliable delivery, queue routing, and the 15-second poll schedule, then imports `app.services.sync` to register tasks. It is the wiring that makes both sync strategies actually run.

FILE: entrypoint.sh

Byte 1: Wait for infrastructure before booting

Builds on: None — starting point

In plain terms:

In Docker, do not start the API until Postgres, Mongo, RabbitMQ, and Elasticsearch accept connections.

The code:

```
targets = [  
    ("postgres", 5432),  
    ("mongodb", 27017),  
    ("rabbitmq", 5672),  
    ("elasticsearch", 9200),  
]
```

What's happening:

Embedded Python loops with socket connects for up to 120 seconds, removing each host as it becomes reachable, and exits nonzero if any stay pending. Actual script uses `socket.create\_connection` with sleep and deadline.

Why it matters:

Prevents race crashes on `docker compose up` where the API starts before its dependencies. Fail-fast with a clear pending list.

Byte 2: Opt-in seed then exec CMD

Builds on: Byte 1

In plain terms:

Seed demo data only on first start when asked, then hand over to the real command.

The code:

```
if [ "${RUN_SEED_ON_START}" = "true" ]; then  
    python scripts/seed.py  
fi  
exec "$@"
```

What's happening:

Runs `scripts/seed.py` only if `RUN\_SEED\_ON\_START=true`, so workers sharing this entrypoint never reseed. `exec "\$@"` replaces the shell with the container CMD.

Why it matters:

Keeps seeding idempotent and opt-in while preserving signals and exit codes via `exec`. Worker and beat containers reuse infra-wait safely.

PUTTING IT TOGETHER

`entrypoint.sh` blocks until infrastructure is reachable, optionally seeds once, then execs the container command. It makes fresh Docker boots deterministic without risking wipes on restarts.

FILE: models.py

Byte 1: Users and orders as source of truth

Builds on: None — starting point

In plain terms:

Defines the only Postgres tables: shoppers and their orders. `No products table — products live in MongoDB.`

The code:

```
class User(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    email: Mapped[str] = mapped_column(String(255), nullable=False, unique=True,
    index=True)
```

What's happening:

`User` has name, unique indexed email, and one-to-many `orders`. `Order` has user FK, timezone-aware dates, indexed status, decimal total, and `updated\_at` auto-bump.

Why it matters:

This is the transactional core. Indexes on email, user, status, and dates support login lookup, history, filtering, and polling.

Byte 2: Order items carry catalog snapshot

Builds on: Byte 1

In plain terms:

Each line item freezes title and price at purchase time so later catalog edits never rewrite history.

The code:

```
# MongoDB product _id as string (no cross-database FK possible).
product_id: Mapped[str] = mapped_column(String(64), nullable=False, index=True)
# Snapshot captured at order time – immune to later catalog changes.
title: Mapped[str] = mapped_column(String(255), nullable=False, default="")
```

What's happening:

Stores Mongo `\_id` as a plain string with no foreign key, plus snapshot `title`, `quantity`, and `unit\_price`. Cascade delete removes items with their order.

Why it matters:

History stays immutable when Mongo prices or titles change. Cross-database joins are impossible, so denormalization is required.

PUTTING IT TOGETHER

`models.py` models users, orders, and snapshot line items with relationships and indexes. Together they enforce atomic order history independent of the live Mongo catalog.

FILE: schemas.py

Byte 1: Product and user contracts

Builds on: None — starting point

In plain terms:

Validates catalog input and shapes user output for the API.

The code:

```
class ProductIn(BaseModel):
    sku: str = Field(min_length=1, max_length=64)
    title: str = Field(min_length=1, max_length=255)
    price: float = Field(gt=0)
```

What's happening:

`ProductIn` enforces non-empty sku and title and positive price, with defaults for tags, attributes, variants, and active. `ProductOut` adds string `id` and `updated\_at`; `UserOut` exposes id, name, email.

Why it matters:

Rejects bad catalog writes at the boundary. Separating In and Out keeps internal `\_id` handling from leaking into requests.

Byte 2: Order creation guards duplicates

Builds on: Byte 1

In plain terms:

An order must have a user and at least one unique product line.

The code:

```
class OrderCreate(BaseModel):
    user_id: int
    items: list[OrderItemIn] = Field(min_length=1)
    @field_validator("items")
    @classmethod
    def no_duplicate_products(cls, items):
        ids = [i.product_id for i in items]
        if len(ids) != len(set(ids)):
            raise ValueError("Duplicate product_id in items")
        return items
```

What's happening:

`OrderItemIn` requires quantity 1–1000. The validator rejects duplicate `product\_id` values in one order.

Why it matters:

Prevents ambiguous totals and double-counting the same product in different lines. Fails fast with 422 before any DB work.

Byte 3: Status and search filter validation

Builds on: Byte 2

In plain terms:

Only the three exact uppercase statuses are accepted everywhere.

The code:

```
class OrderStatusUpdate(BaseModel):
    status: str
    @field_validator("status")
    @classmethod
    def valid_status(cls, v):
        norm = (v or "").strip().upper()
        if norm not in ORDER_STATUSES:
            raise ValueError(f"status must be one of {ORDER_STATUSES}")
        return norm
```

What's happening:

Normalizes to uppercase and checks membership in `ORDER\_STATUSES`. `OrderSearchRequest` does the same for lists, plus pagination, sort literal, and total ranges.

Why it matters:

Keeps Postgres, Elasticsearch keyword field, and UI facets in sync. Bad statuses become 422 instead of silent mismatches.

PUTTING IT TOGETHER

`schemas.py` validates all inbound payloads and shapes outbound order, item, and sync-status responses. It is the contract layer that protects repositories and search from malformed input.

FILE: product_repository.py

Byte 1: Serialize and id-aware lookup

Builds on: None — starting point

In plain terms:

The only product data-access layer. Converts raw Mongo docs to API shapes and handles both ObjectId and string ids.

The code:

```
def serialize_product(doc: dict) -> dict:
    return {
        "id": str(doc.get("_id")),
        "sku": doc.get("sku", ""),
```

What's happening:

Maps `\_id` to string `id` with safe defaults and float coercion. `\_by\_id\_filter` tries `ObjectId`, falling back to raw string for seed and test ids like `m1`.

Why it matters:

Callers always get a stable shape regardless of id type. Without dual handling, seeded string ids would be unfindable.

Byte 2: List, get, and batch fetch for checkout

Builds on: Byte 1

In plain terms:

Serve the storefront and validate carts efficiently.

The code:

```
def list_products(active_only: bool = True, limit: int = 500) -> list[dict]:
    col = get_products_collection()
    filt = {"active": True} if active_only else {}
    return [serialize_product(d) for d in col.find(filt).limit(limit)]
```

What's happening:

Filters active only for storefront or all for admin. `get\_product` fetches one; `get\_products\_by\_ids` does one `\$or` query covering both id forms and returns a map for missing and inactive checks.

Why it matters:

Batch fetch lets order creation validate many lines in one query. Active filtering hides discontinued items from shoppers.

Byte 3: Create, update, and unique SKU

Builds on: Byte 1

In plain terms:

Write catalog changes with timestamps and protect SKU uniqueness.

The code:

```
def ensure_indexes() -> None:
    get_products_collection().create_index("sku", unique=True)
```

What's happening:

`create\_product` builds a doc with `updated\_at`, catches `DuplicateKeyError` as 409. `update\_product` patches only allowed fields, coerces price, bumps timestamp, and never touches orders or ES.

Why it matters:

Enforces catalog integrity at the DB level. Isolating catalog writes proves history snapshots stay untouched.

PUTTING IT TOGETHER

`product\_repository.py` centralizes all Mongo product access: serialization, flexible id lookup, listing, batch validation, writes, and SKU index. No other file should query products directly.

FILE: order_repository.py

Byte 1: Atomic order creation with total

Builds on: None — starting point

In plain terms:

Insert one order plus all its line items as a single all-or-nothing transaction. `PostgreSQL order repository` — atomic transactions, source of truth.`

The code:

```
def create_order(db: Session, user: User, lines: list[dict]) -> Order:
    order = Order(user_id=user.id, status="PENDING", total_amount=0)
    db.add(order)
    db.flush() # assign order.id inside the transaction
```

What's happening:

Flushes to get `order.id`, inserts each `OrderItem` from snapshot lines, sums `qty \* unit\_price`, rounds total, commits, and rolls back on any failure.

Why it matters:

Prevents partial orders with missing items or wrong totals. Postgres remains the consistent source even under errors.

Byte 2: Eager reads and status update

Builds on: Byte 1

In plain terms:

Fetch full order graphs efficiently and change status safely.

The code:

```
def get_order(db: Session, order_id: int) -> Order | None:
    return (
        db.query(Order)
        .options(joinedload(Order.items), joinedload(Order.user))
        .filter(Order.id == order_id)
        .first()
    )
```

What's happening:

Uses `joinedload` to fetch items and user in one round trip, avoiding N+1 queries. `update\_status` sets status, commits, then re-reads the full graph.

Why it matters:

Detail pages need customer plus items together. Re-reading after update ensures the API returns fresh, complete data.

PUTTING IT TOGETHER

`order\_repository.py` owns Postgres order writes and reads: atomic creation with computed totals, eager fetching, and status updates. Callers never write SQL directly.

FILE: documents.py

Byte 1: Build search doc from Postgres only

Builds on: None — starting point

In plain terms:

Flatten an order into the Elasticsearch shape using only committed Postgres rows. `MongoDB` is deliberately NOT consulted here.

The code:

```
def build_order_document(db: Session, order_id: int) -> dict | None:
    order = db.query(Order).filter(Order.id == order_id).first()
    if order is None:
        return None
```

What's happening:

Loads order, user, and items, then builds `order\_id`, ISO dates, status, float total, nested customer, and item snapshots. Returns `None` if missing.

Why it matters:

Guarantees search reflects immutable history, not the live catalog. Catalog renames can never rewrite indexed titles or prices.

Byte 2: Snapshot fields preserve history

Builds on: Byte 1

In plain terms:

Titles and prices come from frozen order items, not live products.

The code:

```
"items": [
    {
        "product_id": it.product_id,
        "title": it.title,
        "quantity": it.quantity,
        "unit_price": float(it.unit_price),
    }
]
```

```
        for it in order.items
    ],
```

What's happening:

Copies each `OrderItem` snapshot verbatim into the nested `items` array. Customer id, name, and email are similarly denormalized.

Why it matters:

This is what the snapshot test asserts. ES stays a true projection of Postgres history.

PUTTING IT TOGETHER

`documents.py` translates one Postgres order into one flat search document from snapshots alone. Both sync strategies and reindex share it, so all ES writes stay identical.

FILE: search_service.py

Byte 1: Sort map and bool query skeleton

Builds on: None — starting point

In plain terms:

Turns UI sort choices and filters into Elasticsearch query pieces. `Elasticsearch-backed admin order search (search path only).`

The code:

```
SORT_MAP = {
    "newest": [{"order_date": {"order": "desc"}}],
    "oldest": [{"order_date": {"order": "asc"}}],
    "total_desc": [{"total_amount": {"order": "desc"}}],
    "total_asc": [{"total_amount": {"order": "asc"}}],
}
```

What's happening:

Maps friendly sort names to ES sort clauses. `build\_es\_query` accumulates `must` for relevance and `filter` for exact constraints, falling back to `match\_all`.

Why it matters:

Separates scored text search from non-scoring filters, which is faster and cacheable. UI sort stays decoupled from ES syntax.

Byte 2: Text, order-id, status, and range handling

Builds on: Byte 1

In plain terms:

Smart search that finds order numbers, customers, products, dates, and totals.

The code:

```
if q and q.strip():
    q = q.strip()
    digits = "".join(ch for ch in q if ch.isdigit())
    if digits and (q.isdigit() or q.upper().startswith("ORD")):
        try:
            filters.append({"term": {"order_id": int(digits)}})
```

What's happening:

Pure numbers or `ORD123` become exact `term` on `order\_id`. Otherwise `multi\_match` searches boosted customer name, email, and item titles. Statuses use `term` or `terms`, dates and totals use `range`.

Why it matters:

One box handles order ids and fuzzy text without confusing them. Boosting makes customer matches rank above item matches.

Byte 3: Paged search with revenue and status aggs

Builds on: Byte 2

In plain terms:

Runs the query and returns hits plus dashboard numbers.

The code:

```
body = {
    "query": build_es_query(q, statuses, date_from, date_to, min_total, max_total),
    "sort": SORT_MAP.get(sort, SORT_MAP["newest"]),
    "from": (page - 1) * size,
    "size": size,
    "aggs": {
        "total_revenue": {"sum": {"field": "total_amount"}},
        "by_status": {"terms": {"field": "status"}},
    },
}
```

What's happening:

Sends query, sort, pagination, and two aggregations. Parses hits with scores and normalizes aggs to revenue float and status counts with `track\_total\_hits`.

Why it matters:

Powers the admin table plus KPI cards in one round trip. Pagination via `from` keeps large result sets safe.

PUTTING IT TOGETHER

`search\_service.py` maps UI filters to bool, multi-match, term, and range queries, executes paged search, and returns hits with revenue and status aggregates. It only reads ES, never Postgres or Mongo.

FILE: sync.py

Byte 1: Dual sync strategies overview

Builds on: None — starting point

In plain terms:

Keeps Elasticsearch eventually consistent with Postgres via fast events plus a polling safety net. `PostgreSQL -> Elasticsearch (TWO strategies).`

The code:

```
def upsert_order_document(order_id: int) -> dict:
    """Shared idempotent upsert used by BOTH strategies + reindex script."""
```

What's happening:

`upsert\_order\_document` builds the doc from Postgres and indexes with `\_id=str(order\_id)`, so repeats overwrite identically. `bulk\_upsert\_orders` does the same for many ids via `bulk`.

Why it matters:

Idempotency makes retries safe. Both strategies and reindex share one write path, preventing drift.

Byte 2: Strategy A dual-write consumer with retry

Builds on: Byte 1

In plain terms:

Consume the RabbitMQ task and copy the latest Postgres row to search, retrying with backoff on failure.

The code:

```
@celery.task(bind=True, name="app.services.sync.sync_order",
             max_retries=settings.SYNC_MAX_RETRIES)
def sync_order(self, order_id: int) -> dict:
    """Strategy A: dual-write consumer (RabbitMQ -> Celery -> ES)."""
```

What's happening:

Ensures the index, upserts, logs `[dual-write]`, and on ES errors retries with exponential `countdown`. ES failures never roll back Postgres.

Why it matters:

Gives near-real-time search without coupling checkout to ES availability. Backoff absorbs transient outages.

Byte 3: Strategy B polling bulk safety net

Builds on: Byte 1

In plain terms:

Every 15 seconds, bulk-sync recently changed orders to catch lost messages or downtime.

The code:

```
@celery.task(bind=True, name="app.services.sync.poll_changed_orders")
def poll_changed_orders(self) -> dict:
    """Strategy B: periodic polling bulk safety net (~every 15s via beat)."""
```

What's happening:

Queries `updated\_at >= cutoff` for the lookback window, bulk-upserts up to 500 ids, logs `[poll]`, and returns checked and synced counts with graceful ES-down handling.

Why it matters:

Covers broker loss, worker downtime, and ES outages. Log prefixes keep the two strategies distinguishable in ops logs.

PUTTING IT TOGETHER

`sync.py` implements both sync paths on one idempotent upsert: event-driven single sync with retries and periodic bulk polling. Together they guarantee eventual consistency without ever consulting MongoDB.

FILE: products.py**Byte 1: Mongo-only catalog endpoints**

Builds on: None — starting point

In plain terms:

Serve and edit products strictly from MongoDB. `Product endpoints — MongoDB ONLY (Screens 1 + 5).`

The code:

```
router = APIRouter(prefix="/api/products", tags=["products"])
@router.get("", response_model=list[ProductOut])
def list_products(all: bool = Query(default=False)):
    """Storefront catalog (active products only) or full catalog for admin."""
    return repo.list_products(active_only=not all)
```

What's happening:

`GET /api/products` returns active only unless `?all=true` for admin. `GET /{id}` returns 404 when missing.

Why it matters:

Enforces the data boundary: storefront never sees Postgres or ES. One flag serves both shop and catalog manager.

Byte 2: Create and update with SKU conflict mapping

Builds on: Byte 1

In plain terms:

Write catalog changes and translate duplicate SKUs to HTTP 409.

The code:

```
@router.post("", response_model=ProductOut, status_code=status.HTTP_201_CREATED)
def create_product(payload: ProductIn):
    try:
        return repo.create_product(payload.model_dump())
    except ValueError as exc:
        raise HTTPException(status_code=409, detail=str(exc))
```

What's happening:

Dumps validated Pydantic payload to dict for the repository. `ValueError` for duplicate sku becomes 409; update also returns 404 when id is unknown.

Why it matters:

Keeps HTTP semantics correct: 201 on create, 409 on conflict, 404 on missing. Catalog edits explicitly never touch order history.

PUTTING IT TOGETHER

`products.py` is a thin Mongo-only CRUD layer that delegates to the repository and maps errors to status codes. It proves catalog writes are isolated from orders and search.

FILE: orders.py

Byte 1: Snapshot order creation after validation

Builds on: None — starting point

In plain terms:

Validate shopper and catalog, freeze price and title, commit atomically, then queue search sync.
`Write path: validate user (PG) + products (MongoDB snapshot).`

The code:

```
found = catalog.get_products_by_ids([i.product_id for i in payload.items])
missing = [i.product_id for i in payload.items if i.product_id not in found]
if missing:
    raise HTTPException(status_code=404, detail=f"Products not found: {missing}")
```

What's happening:

Checks user exists, batch-fetches products, rejects missing 404 and inactive 422, builds snapshot lines, commits via repository, then calls `publish\_order\_sync` only after commit.

Why it matters:

Freezing title and price at order time preserves history. Publishing only after commit prevents syncing rolled-back orders.

Byte 2: Canonical reads from Postgres

Builds on: Byte 1

In plain terms:

Order lists and details always come from Postgres, never search. `Read path (Screen 4): canonical data from PostgreSQL, never ES.`

The code:

```
def _serialize(order) -> OrderOut:
    return OrderOut(
        id=order.id,
        user_id=order.user_id,
        customer_name=order.user.name if order.user else "",
```

What's happening:

Serializes ORM graph to `OrderOut` with customer fields, float totals, and per-line `line\_total`. List supports `user\_id`, `status`, pagination, newest-first; detail and status update return the same shape.

Why it matters:

Makes Postgres the truth for receipts and fulfillment. ES lag or outage never corrupts what the customer sees.

Byte 3: Sync status comparison for UI freshness

Builds on: Byte 2

In plain terms:

Show whether the search copy matches the committed order.

The code:

```
@router.get("/{order_id}/sync", response_model=OrderSyncStatus)
def order_sync_status(order_id: int, db: Session = Depends(get_db)):
    """Compare PostgreSQL updated_at with the Elasticsearch projection."""
```

What's happening:

Fetches ES `updated\_at` by id, compares ISO strings to Postgres `updated\_at`, and returns `in\_sync` only when both exist and match. ES errors yield `None`, not 500.

Why it matters:

Lets the admin detail page display Pending versus In sync. It exposes eventual consistency honestly instead of hiding lag.

PUTTING IT TOGETHER

`orders.py` validates against Mongo, writes snapshots atomically to Postgres, publishes sync afterward, reads canonically from Postgres, and exposes sync freshness. It is the transactional heart linking all three stores safely.

FILE: search.py

Byte 1: Admin search is Elasticsearch only

Builds on: None — starting point

In plain terms:

This endpoint never reads Postgres or Mongo. `Admin search endpoint — Elasticsearch ONLY (never PostgreSQL/MongoDB).`

The code:

```
router = APIRouter(prefix="/api/search", tags=["search"])
@router.post("/orders")
def search_orders_post(payload: OrderSearchRequest):
```

What's happening:

Accepts a validated `OrderSearchRequest` with text, statuses, dates, totals, sort, and pagination, then delegates to `search\_orders` service.

Why it matters:

Enforces the architecture split: operational reads use Postgres, exploratory admin search uses the projection. Search can fail without affecting orders.

Byte 2: Search downtime maps to 503

Builds on: Byte 1

In plain terms:

If the index is down, tell the UI clearly instead of returning wrong data.

The code:

```
except Exception as exc: # ES down -> 503; PostgreSQL data unaffected
    logger.exception("Elasticsearch search failed")
    raise HTTPException(status_code=503, detail=f"Search unavailable: {exc}")
```

What's happening:

Catches any ES exception, logs traceback, and raises 503. The frontend shows an index-unavailable message while Postgres orders stay safe.

Why it matters:

Distinguishes search outage from empty results. Admins know to retry later rather than assuming no orders exist.

PUTTING IT TOGETHER

`api/search.py` validates filters via Pydantic, forwards to the search service, and translates failures to 503. It is a thin, honest proxy that preserves the ES-only boundary.

FILE: users.py

Byte 1: Seeded user listing, no auth

Builds on: None — starting point

In plain terms:

Exposes the fixed demo shoppers for login dropdowns. `User endpoints — PostgreSQL users (seeded, no real auth).`

The code:

```
router = APIRouter(prefix="/api/users", tags=["users"])
@router.get("", response_model=list[UserOut])
def list_users(db: Session = Depends(get_db)):
    return db.query(User).order_by(User.id).all()
```

What's happening:

Queries all Postgres users ordered by id and returns them as `UserOut`. No passwords, tokens, or creation endpoint exist.

Why it matters:

Checkout and storefront need a customer picker without building auth. This keeps the demo focused on orders and search.

PUTTING IT TOGETHER

`users.py` provides one read-only list from Postgres for user selection. It depends only on `get\_db` and schemas, making identity trivial and predictable.

FILE: reindex_orders.py

Byte 1: Full rebuild from Postgres source

Builds on: None — starting point

In plain terms:

Throw away the search index and rebuild every order doc from committed rows. `Reads orders + order\_items snapshots ONLY (never MongoDB).`

The code:

```
def rebuild_index() -> dict:
    """Delete + recreate the index, then bulk-index every PG order."""
    Base.metadata.create_all(bind=engine)
    delete_order_index()
    ensure_order_index()
```

What's happening:

Ensures tables, deletes and recreates the ES index, collects all Postgres order ids, bulk-upserts them, refreshes the index, and returns counts.

Why it matters:

Repairs drift after outages or mapping changes. Using only snapshots guarantees rebuilt history matches Postgres exactly.

PUTTING IT TOGETHER

``reindex_orders.py`` deletes, recreates, and bulk-fills the orders index from Postgres, returning pg versus es counts. Seed and ops reuse it for deterministic recovery.

FILE: seed.py

Byte 1: Fixed reproducible dataset plan

Builds on: None — starting point

In plain terms:

Creates the exact demo data tests and UI expect: 8 users, 42 orders, 24 active plus 1 inactive products. ``Reproducible seed (idempotent, fixed plan — no randomness across runs).``

The code:

```
USERS = [  
    ("John Doe", "john.doe@example.com"),  
    ("Jane Smith", "jane.smith@example.com"),  
    ("Wendy Wireless", "wendy.wireless@example.com"),
```

What's happening:

Hardcodes users and 25 product specs via ``_p`` helpers across peripherals, audio, cables, and office, plus one inactive item. No randomness means every run is identical.

Why it matters:

Search facets, pagination, revenue, and category counts are predictable. Tests can assert exact quotas like 14 per status.

Byte 2: Idempotent wipe, insert, and quota checks

Builds on: Byte 1

In plain terms:

Clear stores, insert users, products, and carefully designed orders, then verify coverage.

The code:

```
def seed_users(db) -> list[User]:  
    db.query(OrderItem).delete()  
    db.query(Order).delete()  
    db.query(User).delete()
```

What's happening:

Deletes Postgres in FK-safe order, inserts Mongo products with unique sku index, builds dated orders with snapshot items and totals, commits, then asserts counts for statuses, dates, totals, per-user minimums, and averages.

Why it matters:

Asserts catch broken seeds immediately. Date and total spreads guarantee every filter and sort has meaningful hits.

Byte 3: Rebuild search then prove snapshots

Builds on: Byte 2

In plain terms:

Index the orders, then rename the catalog mouse to prove history never changes.

The code:

```
def snapshot_test_rename_mouse():
    col = get_products_collection()
    col.update_one(
        {"title": "Wireless Mouse"},
        {"$set": {"title": "Wireless Mouse Pro", "price": 34.99,
                  "updated_at": datetime.now(timezone.utc)}},
    )
```

What's happening:

Calls `rebuild\_index` from Postgres, then updates only the Mongo title and price. `already\_seeded` skips when 8 users, 40 plus orders, and 24 products exist, and treats unreachable DBs as seeded to avoid wipes.

Why it matters:

Demonstrates the core invariant: catalog shows Pro, old orders and ES docs still show original title and price. Skipping prevents data loss on restarts.

PUTTING IT TOGETHER

`seed.py` wipes and recreates users, products, and quota-checked orders, rebuilds ES from Postgres, then renames the mouse to prove snapshot isolation. It is skipped when data already exists.

FILE: test_level2.py

Byte 1: Mocked infra harness with fixtures

Builds on: None — starting point

In plain terms:

Runs the whole API against in-memory SQLite, mongomock, and fake ES. `Level 2 tests (mocked infra: SQLite + mongomock + fake ES).`

The code:

```
engine = create_engine("sqlite://", connect_args={"check_same_thread": False},
poolclass=StaticPool)
TestingSession = sessionmaker(bind=engine, autoflush=False, autocommit=False)
```

What's happening:

Creates a shared in-memory SQLite engine, overrides `get\_db`, builds a `TestClient`, and defines mouse, keyboard, and inactive fixtures. `clean\_all` recreates tables and Mongo per test and patches both collection getters.

Why it matters:

Tests run with `pytest -q` and no Docker. Patching both import paths prevents real Mongo leaks.

Byte 2: Catalog and snapshot order tests

Builds on: Byte 1

In plain terms:

Verifies active filtering, CRUD, totals, rollbacks, and history immutability.

The code:

```
def test_order_snapshots_title_and_price_and_total():
    p1, p2 = _patch_mongo()
    with p1, p2, patch("app.api.orders.publish_order_sync", return_value=True) as pub:
```

What's happening:

Asserts storefront hides inactive, admin sees all plus create and update. Order test checks 201, PENDING, computed total, frozen title, publish called, and DB snapshot. Missing and inactive products return 404 and 422 with no publish and zero orders.

Why it matters:

Locks in the snapshot contract and atomicity. Rename test proves PG and ES docs built from PG stay on old title and price.

Byte 3: Search translation and sync strategies

Builds on: Byte 1

In plain terms:

Checks ES query building, endpoint validation, idempotent sync, retries, and polling.

The code:

```
def test_search_uses_bool_multimatch_range_and_aggs():
    from app.services import search_service
    captured = {}
```

What's happening:

Fake ES captures the body and asserts multi-match, terms, range, bool, and revenue plus status aggs. Dual-write test runs `sync\_order` twice to same id and forces ES boom to assert retry without Postgres rollback. Poll test fakes bulk and asserts the new order is synced.

Why it matters:

Proves search syntax, pagination, status updates, sync freshness, and both sync strategies without live infra. Failures here mean UI filters or recovery broke.

PUTTING IT TOGETHER

`test\_level2.py` mocks all three stores, seeds minimal fixtures per test, and covers catalog, snapshot orders, status and sync endpoints, search translation, and both sync paths. It is the executable specification for Level 2 behavior.

FILE: index.html

Byte 1: SPA shell and mount point

Builds on: None — starting point

In plain terms:

The single HTML page that hosts the entire Vue marketplace.

The code:

```
<div id="app"></div>
<script type="module" src="/src/main.js"></script>
```

What's happening:

Provides charset, viewport, title `Level 2 E-Commerce — Orders & Search`, an empty `#app` div, and a module script loading `main.js`.

Why it matters:

Vue replaces `#app` at runtime. Without this mount and entry script, no view renders.

PUTTING IT TOGETHER

`index.html` is the minimal browser shell: metadata, title, mount div, and module entry. Vite serves it in dev and bundles it for nginx in production.

FILE: vite.config.js

Byte 1: Vue plugin, port, and API proxy

Builds on: None — starting point

In plain terms:

Configures local development so `/api` calls reach FastAPI without CORS issues.

The code:

```
export default defineConfig({
  plugins: [vue()],
  server: {
    port: 5173,
    proxy: {
      '/api': 'http://localhost:8002'
    }
  }
})
```

What's happening:

Enables the Vue compiler, fixes dev port to 5173, and proxies `/api` to FastAPI on 8002. Same-origin in dev means cookies and CORS stay simple.

Why it matters:

Matches the backend CORS list and `client.js` relative URLs. Frontend code needs no absolute host in development.

PUTTING IT TOGETHER

`vite.config.js` wires Vue compilation, dev port, and API proxy in six lines. It is the glue that lets the SPA call FastAPI as same-origin locally.

FILE: main.js

Byte 1: Create app with router and styles

Builds on: None — starting point

In plain terms:

Starts Vue, installs navigation, loads global CSS, and mounts to the page.

The code:

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router/index.js'
import './styles.css'
createApp(App).use(router).mount('#app')
```

What's happening:

Imports root component, router with all routes, and global stylesheet, then mounts onto `#app` from `index.html`.

Why it matters:

Without `use(router)`, links do nothing. Without mount, nothing renders. This is the frontend counterpart to backend `main.py`.

PUTTING IT TOGETHER

`main.js` composes App, router, and styles into a running SPA in four imports and one mount. All views and APIs flow from this entry.

FILE: client.js

Byte 1: Shared axios instance

Builds on: None — starting point

In plain terms:

One HTTP client for the whole UI with same-origin default and timeout. `Same-origin: Vite dev proxy and any reverse proxy forward /api to FastAPI.`

The code:

```
const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || '',
  timeout: 15000
})
```

What's happening:

Uses relative base URL so dev proxy and nginx handle routing, falling back to `VITE\_API\_URL` when set. Fails after 15 seconds instead of hanging.

Why it matters:

Avoids hardcoding hosts across environments. Every API helper shares timeout and base path consistently.

Byte 2: Typed resource helpers

Builds on: Byte 1

In plain terms:

Small wrappers for users, products, orders, and search so views stay clean.

The code:

```
export const ordersApi = {
  create: (payload) => api.post('/api/orders', payload).then(r => r.data),
  list: (params = {}) => api.get('/api/orders', { params }).then(r => r.data),
  get: (id) => api.get(`/api/orders/${id}`).then(r => r.data),
}
```

What's happening:

Exposes list, get, create, update, status, sync-status, and search POST, unwrapping `r.data`. Product list supports `{all:true}` for admin; order list passes query params.

Why it matters:

Views call domain verbs instead of raw URLs. Changing an endpoint means editing one helper, not seven views.

PUTTING IT TOGETHER

`client.js` owns the axios instance and four resource objects. It isolates HTTP details so Vue components focus on state and rendering.

FILE: store.js

Byte 1: Reactive user, cart, and persistence

Builds on: None — starting point

In plain terms:

Frontend-only cart and login with localStorage survival across reloads. `Cart lives in frontend/local state only — no cart database.`

The code:

```
const saved = JSON.parse(localStorage.getItem('l2-cart') || '[]')
export const store = reactive({
  user: savedUser,
  setUser(u) {
    this.user = u
    localStorage.setItem('l2-user', JSON.stringify(u))
  },
})
```

What's happening:

Loads saved cart and user, makes them reactive, and persists on every mutation. `clearCart` splices in place to keep reactivity.

Why it matters:

No backend cart table is needed. Refreshing the page does not lose the shopper's work.

Byte 2: Cart mutations and derived totals

Builds on: Byte 1

In plain terms:

Add, change quantity, remove, and auto-compute count and price.

The code:

```
export function addToCart(product, qty = 1) {
  const line = store.cart.find(l => l.id === product.id)
  if (line) line.qty += qty
  else store.cart.push({ id: product.id, title: product.title, price: product.price,
    category: product.category || '', qty })
}
```

What's happening:

Merges same product lines, stores id, title, price snapshot, and qty. `setQty` removes at zero, `removeFromCart` splices, and `cartCount` and `cartTotal` reduce over lines.

Why it matters:

Keeps cart logic in one tested place. Header badge and checkout summary stay in sync automatically.

Byte 3: Browser order history tracking

Builds on: Byte 1

In plain terms:

Remember placed order ids locally for My Orders fallback.

The code:

```
export function recordOrder(id) {
  const ids = JSON.parse(localStorage.getItem('l2-orders') || '[]')
  if (!ids.includes(id)) {
    ids.unshift(id)
    localStorage.setItem('l2-orders', JSON.stringify(ids.slice(0, 50)))
  }
}
```

What's happening:

Prepends new id, dedupes, caps at 50. `recentOrderIds` reads the list for fetching via `GET /api/orders/:id`.

Why it matters:

Lets shoppers see receipts even without server-side history query. Complements Postgres user history, not replaces it.

PUTTING IT TOGETHER

`store.js` provides reactive user and cart with persistence, mutation helpers, computed totals, and local order-id history. It is the frontend's lightweight state layer with no backend dependency.

FILE: productArt.js

Byte 1: Keyword-to-artwork type table

Builds on: None — starting point

In plain terms:

Picks an illustration style from product words without any network images. `Frontend-only product artwork mapper (UI layer only — no backend changes).`

The code:

```
const TYPES = [
  { key: 'mouse', test: /mouse/i, bg: ['#1e3a8a', '#3b82f6'], glyph: 'mouse' },
  { key: 'keyboard', test: /keyboard/i, bg: ['#4c1d95', '#8b5cf6'], glyph: 'keyboard' },
]
```

What's happening:

Ordered regex list matches title, category, and tags against 16 types, each with gradient colors and glyph name. First match wins.

Why it matters:

Gives every product distinct, never-broken art with zero image hosting. Order matters because hub must not swallow cable patterns incorrectly.

Byte 2: Inline SVG glyphs as data URIs

Builds on: Byte 1

In plain terms:

Builds a self-contained image URL from gradients plus white line icons.

The code:

```
export function productArt(product) {
  const t = typeFor(product)
  const svg = `
```

What's happening:

`GLYPHS` holds stroke-based icon paths. `productArt` wraps the matched glyph in a gradient with decorative circles and URI-encodes it. `artByTitle` handles cart lines with only titles; `categoryArt` maps four categories to glyphs.

Why it matters:

Data URIs need no server and never 404. Unknown products fall back to chip art, so UI never shows a broken image.

PUTTING IT TOGETHER

`productArt.js` matches product text to a type, renders its glyph on a gradient SVG, and returns a data URI, with title-only and category helpers. It is pure UI polish isolated from backend logic.

FILE: index.js

Byte 1: Seven routes for shop and admin

Builds on: None — starting point

In plain terms:

Maps URLs to the store, checkout, orders, search, and catalog screens.

The code:

```
import Storefront from '../views/Storefront.vue'
import ProductDetail from '../views/ProductDetail.vue'
import Checkout from '../views/Checkout.vue'
```

What's happening:

Imports seven view components and creates a `createWebHistory` router with `/`, `/products/:id`, `/checkout`, `/orders`, `/admin`, `/admin/orders/:id`, and `/catalog`.

Why it matters:

Defines the navigation model: shop flow versus admin search versus catalog manager. History mode gives clean URLs without hashes.

PUTTING IT TOGETHER

`router/index.js` declares all routes in one table and exports the router for `main.js`. Adding a screen means adding one import and one route entry.

FILE: App.vue

Byte 1: Global header, search, and navigation

Builds on: None — starting point

In plain terms:

The persistent marketplace shell with branding, search box, login picker, and cart badge.

The code:

```
<router-link class="logo" to="/">
  <span class="logo-mark">V</span>
  <span class="logo-text"><b>VoltEdge</b><small>Electronics Market</small></span>
</router-link>
```

What's happening:

Template renders utility strip, sticky topbar with header search input, user select, Shop, Checkout, My Orders, Admin, Catalog links, cart pill with `cartCount`, `router-view`, and footer noting Mongo, Postgres, and ES roles.

Why it matters:

One shell serves all screens. Header search pushes `?q=` to storefront; login persists via store for checkout.

Byte 2: User bootstrap and search navigation

Builds on: Byte 1

In plain terms:

Loads shoppers once and defaults to the first one.

The code:

```
onMounted(async () => {  
  users.value = await usersApi.list().catch(() => [])  
  if (!store.user && users.value.length) {  
    store.setUser(users.value[0])  
    userId.value = users.value[0].id  
  }  
})
```

What's happening:

Fetches users, tolerates API-down with empty list, auto-selects first user if none saved, and `persistUser` stores dropdown choice. `goSearch` routes to `/` with query.

Why it matters:

Checkout always has a customer without forcing login. Graceful fallback keeps the shell usable when backend is stopped.

PUTTING IT TOGETHER

`App.vue` combines branded header, user and cart state, global search routing, and footer into the root layout. Every view renders inside its `router-view` with live cart count.

FILE: Storefront.vue

Byte 1: Hero and category tiles from live catalog

Builds on: None — starting point

In plain terms:

Landing experience with promo hero, category counts, and under-\$25 deals computed from Mongo products.

The code:

```
const categories = computed(() => {  
  const map = {}  
  for (const p of products.value) {  
    if (!p.category) continue  
    map[p.category] = map[p.category] || { name: p.category, count: 0, art:  
categoryArt(p.category) }  
    map[p.category].count += 1  
  }  
})
```

What's happening:

Counts products per category with generated art, plus `dealPicks` for cheap items. Hero stats show live product and category totals.

Why it matters:

Proves the catalog is live, not hardcoded. Category tiles drive filtering without extra API calls.

Byte 2: Client-side filter, sort, and deals toggle

Builds on: Byte 1

In plain terms:

Search, category, deals, and sorting happen instantly in the browser.

The code:

```
const filtered = computed(() => {
  const needle = q.value.trim().toLowerCase()
  let list = products.value.filter(p =>
    (!category.value || p.category === category.value) &&
    (!onlyDeals.value || Number(p.price) < 25) &&
    (!needle || [p.title, p.description, p.sku, (p.tags || []).join(' ')].join(' ')
    ).toLowerCase().includes(needle)))
}
```

What's happening:

Filters by category, under-\$25 flag, and free-text across title, description, sku, tags. Sorts by price or name; featured keeps server order. Watches route query for header search.

Why it matters:

Fast faceting without backend load. URL query sync lets header search and shared links work.

Byte 3: Catalog load with loading and empty states

Builds on: Byte 2

In plain terms:

Fetches products and users once, handling failure gracefully.

The code:

```
onMounted(async () => {
  try {
    const [prods, us] = await Promise.all([productsApi.list({}), usersApi.list()])
    products.value = prods
    users.value = us
  }
})
```

What's happening:

Parallel fetch of active products and users, auto-login to first user, skeleton grid while loading, error banner if API down, empty card with plug emoji if no matches, and Add to Cart per card.

Why it matters:

Covers all data states instead of blank screens. Parallel loads minimize time to interactive.

PUTTING IT TOGETHER

`Storefront.vue` loads the active Mongo catalog, derives categories and deals, filters and sorts locally with URL sync, and renders cards with art and cart actions. It is Screen 1 of the demo.

FILE: ProductDetail.vue**Byte 1: Single product fetch by route id**

Builds on: None — starting point

In plain terms:

Shows one catalog item based on the URL. Back link returns to the store.

The code:

```
onMounted(async () => {  
  try {  
    product.value = await productsApi.get(route.params.id)  
  } catch (e) {  
    error.value = 'Product not found.'  
  }  
})
```

What's happening:

Reads `:id` from the route, calls `GET /api/products/:id`, stores result, and shows an error banner on 404 instead of crashing.

Why it matters:

Deep links from storefront, deals, and history all land here. Graceful 404 handles deleted or bad ids.

Byte 2: Spec table, quantity, and cart actions

Builds on: Byte 1

In plain terms:

Displays art, price, tags, attributes, variants, and lets shoppers choose quantity.

The code:

```
<span class="qty">
  <button @click="qty = Math.max(1, qty - 1)">-</button>{{ qty }}<button @click="qty +=
1">+</button>
</span>
<button class="btn" @click="addToCart(product, qty); added = true">Add to Cart</button>
```

What's happening:

Renders category tag, description, price, sku plus dynamic attribute rows and variant names, with qty stepper floored at 1. Add calls shared `addToCart` and shows confirmation; checkout link continues flow.

Why it matters:

Centralizes variant display without variant selection complexity. Reuses the same cart helper as the grid for consistent behavior.

PUTTING IT TOGETHER

`ProductDetail.vue` fetches one Mongo product by id and presents art, specs, quantity stepper, and cart actions. It bridges browsing and checkout with minimal state.

FILE: Checkout.vue

Byte 1: Cart review with quantity editing

Builds on: None — starting point

In plain terms:

Shows what the shopper will buy with live totals. `Screen 2 · MongoDB validation → PostgreSQL order.`

The code:

```
<div v-for="l in store.cart" :key="l.id" class="cart-line">
  
```

What's happening:

Iterates reactive cart lines with art by title, per-each price, stepper calling `setQty`, line total, and remove button. Empty cart shows empty card unless a confirmation message exists.

Why it matters:

Shoppers correct mistakes before paying. Reactive store means edits instantly update header count and summary.

Byte 2: Snapshot order placement

Builds on: Byte 1

In plain terms:

Converts cart lines to an order with server-validated prices. `Prices are re-validated against the live catalog.`

The code:

```
async function placeOrder() {
  placing.value = true
  msg.value = ''
  try {
    const order = await ordersApi.create({
      user_id: Number(userId.value),
      items: store.cart.map(l => ({ product_id: l.id, quantity: l.qty }))
    })
  }
```

What's happening:

Posts only ids and quantities; backend snapshots title and price, computes total, commits to Postgres, queues ES sync. On success records id locally, clears cart, shows order number; on failure shows backend detail. Button disables without user, empty cart, or while placing.

Why it matters:

Frontend never trusts its displayed prices. This is where snapshot isolation is exercised from the UI.

PUTTING IT TOGETHER

`Checkout.vue` reviews and edits the local cart, posts ids plus quantities with chosen user, records the new order, and clears the cart. It demonstrates Mongo validation to Postgres ordering end to end.

FILE: MyOrders.vue

Byte 1: Dual-source order history

Builds on: None — starting point

In plain terms:

Shows your receipts from Postgres user history plus this browser's recent ids. `PostgreSQL · canonical order reads.`

The code:

```

if (store.user) {
  try {
    for (const o of await ordersApi.list({ user_id: store.user.id, size: 50 })) {
      if (!seen.has(o.id)) {
        seen.add(o.id)
        out.push(o)
      }
    }
  } catch (e) { /* fall back to local IDs below */ }
}

```

What's happening:

First fetches up to 50 Postgres orders for the current user, deduping by id. Then fetches each `recentOrderIds` from localStorage via `GET /:id`, skipping already seen and vanished orders.

Why it matters:

Covers both logged-in history and guest-browser receipts. Postgres failures still leave local ids usable.

Byte 2: Order cards with detail links

Builds on: Byte 1

In plain terms:

Compact receipt cards with status, date, preview items, and total.

The code:

```

<div v-for="o in orders" :key="o.id" class="pcard">
  <div class="body">
    <div class="row" style="padding-top:0">
      <b>Order #{{ o.id }}</b>
      <span class="status" :class="o.status">{{ o.status }}</span>
    </div>
  </div>
</div>

```

What's happening:

Shows skeleton while loading, empty card with start-shopping link if none, else status pill, date, item count, up to three item rows with art, total, and Details link to admin order page.

Why it matters:

Gives shoppers proof of snapshot prices and a path to canonical Postgres detail. Status colors match admin for consistency.

PUTTING IT TOGETHER

`MyOrders.vue` merges Postgres user orders and local recent ids into deduped cards with totals and detail links. It reads only canonical Postgres data, never search.

FILE: AdminSearch.vue

Byte 1: Elasticsearch-only filter form

Builds on: None — starting point

In plain terms:

Admin controls for full-text and faceted order search. `Screen 3 · Elasticsearch ONLY.`

The code:

```
<input class="input" v-model="f.q" placeholder="Search customers, products, orders..."
style="min-width:260px;flex:1" @keyup.enter="search(1)" />
```

What's happening:

Reactive `f` holds q, status checkboxes, date range, min and max totals, sort, and size. Enter or Search button triggers `search(1)`; status list is fixed to three values.

Why it matters:

Exposes every backend filter without page reloads. Checkbox model maps directly to the API statuses array.

Byte 2: Search request, KPIs, and results table

Builds on: Byte 1

In plain terms:

Posts filters to search, shows revenue and counts, pages through hits.

The code:

```
const out = await searchApi.orders({
  q: f.q || null,
  statuses: f.statuses.length ? f.statuses : null,
  date_from: f.date_from || null,
  date_to: f.date_to ? f.date_to + 'T23:59:59' : null,
```

What's happening:

Nulls empty filters, extends `date\_to` to end of day, sends sort, page, size, then stores results, aggs, and total. KPIs show filtered revenue, per-status counts, total hits; table links each `order\_id` to detail. Prev and Next paginate; 503 shows index-down message distinguishing outage from no matches.

Why it matters:

One call powers both table and dashboard. Honest 503 messaging preserves trust when ES lags Postgres.

PUTTING IT TOGETHER

`AdminSearch.vue` collects text, status, date, total, and sort filters, posts to ES-only search, renders KPIs plus paged results, and handles index outages explicitly. It never queries Postgres directly.

FILE: AdminOrderDetail.vue

Byte 1: Canonical Postgres order with sync freshness

Builds on: None — starting point

In plain terms:

Shows the true receipt plus whether search has caught up. `Screen 4 · PostgreSQL (canonical).`

The code:

```
async function load() {
  try {
    order.value = await ordersApi.get(id)
    nextStatus.value = order.value.status
    sync.value = await ordersApi.syncStatus(id).catch(() => null)
  }
}
```

What's happening:

Loads `GET /api/orders/:id` for customer, dates, snapshot lines with line totals, and grand total, plus `GET /sync` for `in\_sync` versus Pending and PG versus ES timestamps. Missing id shows Postgres-not-found error.

Why it matters:

Receipt truth comes from Postgres even when ES is stale. Sync badge makes eventual consistency visible instead of confusing.

Byte 2: Fulfillment status update flow

Builds on: Byte 1

In plain terms:

Change PENDING, PROCESSING, or SHIPPED and queue search sync.

The code:

```
async function updateStatus() {
  msg.value = ''
}
```

```
order.value = await ordersApi.updateStatus(id, nextStatus.value)
msg.value = 'Status saved to PostgreSQL and sync queued → search index updates
shortly.'
```

What's happening:

Dropdown defaults to current status, PATCH saves to Postgres then publishes sync, confirmation explains ES lag, and sync status is re-fetched. Receipt labels snapshot lines as `snapshot · qty × price`.

Why it matters:

Enforces Postgres-first fulfillment. UI copy sets correct expectations about search delay.

PUTTING IT TOGETHER

`AdminOrderDetail.vue` renders canonical Postgres receipt with sync comparison and provides status updates that commit to Postgres before syncing. It links back to search while staying truthful about lag.

FILE: CatalogAdmin.vue

Byte 1: Mongo-only catalog grid with snapshot proof

Builds on: None — starting point

In plain terms:

Manage products knowing history is safe. `Screen 5 · MongoDB products. Catalog edits change MongoDB only — historical orders are never rewritten.`

The code:

```
async function load() {
  products.value = await productsApi.list({ all: true }).catch(() => [])
}
```

What's happening:

Loads full catalog including inactive with `?all=true`, filters locally by title, sku, category, and renders art cards with active badges and edit buttons. Header copy cites Wireless Mouse Pro versus old Wireless Mouse.

Why it matters:

Proves snapshot isolation in the UI itself. Admin sees all items while storefront hides inactive.

Byte 2: Create and edit form preserving variants

Builds on: Byte 1

In plain terms:

Add or edit catalog fields and save to Mongo only.

The code:

```
const payload = {
  sku: form.sku, title: form.title, description: form.description, price:
  Number(form.price),
  category: form.category, tags: form.tags.split(',').map(t =>
  t.trim()).filter(Boolean),
```

What's happening:

Form holds sku, title, description, price, category, comma tags, brand, active. `startCreate` blanks it; `startEdit` fills from product including tags join and brand. Save builds attributes and preserves variants on edit or seeds Standard on create, then reloads and confirms `Saved to MongoDB. Historical orders untouched.`

Why it matters:

Simplifies variant handling while protecting history. Comma parsing keeps tag UX friendly without backend changes.

PUTTING IT TOGETHER

`CatalogAdmin.vue` lists the full Mongo catalog with local filter, edits or creates via a shared form, preserves variants, and reloads with confirmation. It demonstrates catalog isolation from order history.

FILE: styles.css

Byte 1: Design tokens and base layout

Builds on: None — starting point

In plain terms:

Defines the VoltEdge look: colors, radius, shadows, fonts, and page frame. `VoltEdge` — original premium electronics marketplace theme (UI only).`

The code:

```
:root {
  --bg: #eef1f6;
  --surface: #ffffff;
  --ink: #0e1730;
  --brand: #16245c;
  --accent: #ff5c1a;
```

What's happening:

CSS variables centralize palette, status colors, radius, and shadows. Base rules set box sizing, smooth scroll, font stack, background, and responsive container, header, and footer.

Why it matters:

One token change rethemes the app. Layout classes give every view consistent spacing without per-file CSS.

Byte 2: Commerce components and status system

Builds on: Byte 1

In plain terms:

Styles for hero, categories, cards, cart, tables, KPIs, and order states.

The code:

```
.status.PENDING { background: var(--warn-bg); color: var(--warn); }  
.status.PROCESSING { background: var(--info-bg); color: var(--info); }  
.status.SHIPPED { background: var(--ok-bg); color: var(--ok); }
```

What's happening:

Provides hero grid, category tiles, deals strip, toolbar, product grid, receipt split, cart lines, quantity steppers, results tables, KPI cards, alerts, skeletons with shimmer, and source badges for ES, PG, and Mongo. Media queries collapse hero and split below 960px and form grid below 560px.

Why it matters:

Status pills and source badges make data provenance visual. Responsive rules keep admin tables and checkout usable on mobile.

PUTTING IT TOGETHER

`styles.css` supplies tokens, layout, commerce components, status colors, and responsive behavior in one global sheet. All Vue views share it, so branding and state signaling stay uniform.